CAMPUSCONNECT EVENT MANAGER

A beginner-friendly Python project for GitHub

Project Type: Command-line application

Technology: Python 3 • JSON file storage

PROJECT DOCUMENTATION

Includes project overview, objectives, features, complete source code, setup instructions, sample run, testing checklist, README starter, and GitHub upload steps.

1. Project Overview

CampusConnect Event Manager is a simple application for managing college events and student registrations. Users can add events, view available events, register a student for an event, and view registration records. Data is saved locally in JSON files so it remains available after the program closes.

2. Problem Statement

College event registrations are often managed using paper lists or scattered messages. This can make it difficult to track events and participants. CampusConnect provides a simple centralized tool for recording event details and student registrations.

3. Objectives

• Create and store college event details.

• Display all events with their IDs and dates.

• Register a student for an event.

• Prevent duplicate registration of the same student for the same event.

• Display all registrations for a selected event.

• Practice Python functions, lists, dictionaries, loops, validation, and file handling.

4. Main Features

Feature	Description
Add event	Save event name, date, and venue.
View events	List saved events with unique IDs.
Register student	Record student name and ID for an event.
View registrations	Display students registered for a selected event.
Local storage	Keep events and registrations in JSON files.

5. Requirements

Software: Python 3.9 or later and a code editor such as VS Code. **Libraries:** Built-in Python libraries only; no extra packages are needed.

6. Folder Structure

```
campusconnect-event-manager/
■■■ event_system.py
■■■ events.json             # created automatically
■■■ registrations.json      # created automatically
■■■ README.md
■■■ .gitignore
```

7. Complete Source Code

Save the following code as **event_system.py**.

```
import json
import os

EVENT_FILE = "events.json"
REG_FILE = "registrations.json"


def load_data(filename):
    if not os.path.exists(filename):
        return []
    try:
        with open(filename, "r", encoding="utf-8") as file:
```

```
            return json.load(file)
    except (json.JSONDecodeError, OSError):
        print(f"Warning: Could not read {filename}. Starting empty.")
        return []


def save_data(filename, data):
    with open(filename, "w", encoding="utf-8") as file:
        json.dump(data, file, indent=4)


def add_event(events):
    name = input("Event name: ").strip()
    event_date = input("Event date (YYYY-MM-DD): ").strip()
    venue = input("Venue: ").strip()
    if not name or not event_date or not venue:
        print("All fields are required.")
        return

    next_id = max((event["id"] for event in events), default=0) + 1
    events.append({"id": next_id, "name": name, "date": event_date, "venue": venue})
    save_data(EVENT_FILE, events)
    print(f"Event added successfully. Event ID: {next_id}")


def view_events(events):
    if not events:
        print("No events available.")
        return
    print("\\n--- Campus Events ---")
    for event in events:
        print(f"ID: {event['id']} | {event['name']} | {event['date']} | Venue: {event['venue']}")


def register_student(events, registrations):
    if not events:
        print("Add an event before registering students.")
        return
    view_events(events)
    try:
        event_id = int(input("Enter event ID: "))
    except ValueError:
        print("Event ID must be a number.")
        return
    if not any(event["id"] == event_id for event in events):
        print("Event ID not found.")
        return

    student_name = input("Student name: ").strip()
    student_id = input("Student ID: ").strip()
    if not student_name or not student_id:
        print("Student name and ID are required.")
        return
    if any(r["event_id"] == event_id and r["student_id"] == student_id for r in registrations):
        print("This student is already registered for this event.")
        return

    registrations.append({"event_id": event_id, "student_name": student_name, "student_id": student_id})
    save_data(REG_FILE, registrations)
    print("Registration completed successfully!")


def view_registrations(events, registrations):
    if not events:
        print("No events available.")
        return
    view_events(events)
    try:
        event_id = int(input("Enter event ID to view registrations: "))
    except ValueError:
        print("Event ID must be a number.")
        return
    event = next((e for e in events if e["id"] == event_id), None)
    if event is None:
        print("Event ID not found.")
        return

    print(f"\\nRegistrations for {event['name']}:")
    found = False
    for item in registrations:
        if item["event_id"] == event_id:
            print(f"{item['student_name']} ({item['student_id']})")
            found = True
    if not found:
        print("No students registered yet.")
```

```
def main():
    events = load_data(EVENT_FILE)
    registrations = load_data(REG_FILE)
    while True:
        print("\\n===== CampusConnect Event Manager =====")
        print("1. Add event")
        print("2. View events")
        print("3. Register student")
        print("4. View event registrations")
        print("5. Exit")
        choice = input("Choose an option: ").strip()
        if choice == "1":
            add_event(events)
        elif choice == "2":
            view_events(events)
        elif choice == "3":
            register_student(events, registrations)
        elif choice == "4":
            view_registrations(events, registrations)
        elif choice == "5":
            print("Thank you for using CampusConnect!")
            break
        else:
            print("Invalid choice. Select 1 to 5.")


if __name__ == "__main__":
    main()
```

8. How to Run

• Install Python 3.9 or later if it is not already installed.

• Create a folder named campusconnect-event-manager.

• Save the code as event_system.py inside the folder.

• Open a terminal in that folder.

• Run: python event_system.py (or python3 event_system.py).

• Use menu option 1 to add an event, then option 3 to register a student. JSON files are created automatically.

9. Sample Run

```
===== CampusConnect Event Manager =====
1. Add event
2. View events
3. Register student
4. View event registrations
5. Exit
Choose an option: 1
Event name: Tech Symposium
Event date (YYYY-MM-DD): 2026-11-15
Venue: Main Auditorium
Event added successfully. Event ID: 1

Choose an option: 3
ID: 1 | Tech Symposium | 2026-11-15 | Venue: Main Auditorium
Enter event ID: 1
Student name: Arun
Student ID: 24CS101
Registration completed successfully!
```

10. Testing Checklist

Test	Action	Expected result
1	Add event with all fields	Event is saved with an ID.
2	View events	Saved event details appear.
3	Register a student	Registration is saved.
4	Register same student ID twice for same event	Duplicate registration is rejected.
5	Enter unknown event ID	An error message is displayed.
6	Exit with option 5	Program ends normally.

11. README.md Starter

````
# CampusConnect Event Manager

A beginner-friendly Python application to manage college events and student registrations.

## Features
- Add and view campus events
- Register students for events
- Prevent duplicate registrations
- View registrations for each event
- Save data locally in JSON files

## Requirements
- Python 3.9 or later
- No external packages required

## Run
```bash
python event_system.py
```

## Future Improvements
- Add event capacity limits
- Search and cancel registrations
- Export participant lists to CSV
````

```
- Build a web interface with Flask
```

12. Upload to GitHub

• Sign in to GitHub and create a repository named campusconnect-event-manager.

• Upload event_system.py and README.md. You can also upload this PDF as documentation.

• Do not upload real or private student information in registrations.json.

• Alternatively, run these commands in the project folder:

```
git init
git add event_system.py README.md .gitignore
git commit -m "Add CampusConnect Event Manager"
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Suggested .gitignore

```
events.json
registrations.json
__pycache__/
*.pyc
.env
```

13. Conclusion

CampusConnect Event Manager is a practical beginner project that demonstrates Python programming, data structures, input validation, and JSON file handling. It can be extended with a graphical interface, event capacity management, search, cancellation, and CSV reports.